

Алгоритмы и структуры данных

Графовые · Строковые алгоритмы · Парадигмы разработки

Учебный год 2025–2026 · 15 билетов

Как устроен билет. Каждый билет разложен по единому формату:

Постановка — что решаем
Алгоритм — шаги/псевдокод
Идея доказательства
Сложность $T_{wc}(n)$ с обоснованием
Пример — трассировка + SVG

Идея — ключевое наблюдение
Теорема · корректность
Строгое доказательство — инвариант/индукция
Ограничения применимости
На экзамене — типичные ловушки

Требования экзамена к ответам разделов 1–2: обоснование асимптотики $T_{wc}(n)$, комментарий об ограничениях применимости, пошаговая иллюстрация на примере.

Содержание

Билет 1. Кратчайшие пути между всеми парами — Флойда–Уоршелла	2
Билет 2. Максимальный поток в сети — Форд–Фалкерсон / Эдмондс–Карп / Диниц	4
Билет 3. Паросочетания в графе. Базовый алгоритм / Хопкрофта–Карпа	6
Билет 4. Правильная раскраска графа — жадный алгоритм	8
Билет 1. Точное вхождение строки в текст — Кнута–Морриса–Пратта / Бойера–Мура–Хорспула	10
Билет 2. Приближённое вхождение — редакционное расстояние Левенштейна / Дамерау–Левенштейна	12
Билет 3. Вхождение множества строк — автомат Ахо–Корасик	14
глубина текущей вершины) $\Rightarrow O(n)$ переходов, плюс $O(z)$ на выдачу всех	14
Билет 4. Специализированные сортировки строк — String Merge Sort / String Quick Sort / MSD Radix Sort	16
Билет 5. Кодирование и сжатие — Run-Length Encoding / код Хаффмана / код LZW	18
Билет 1. Жадный алгоритм. Доказательство оптимальности	20
Билет 2. Динамическое программирование: подходы. Связь с «разделяй-и-властвуй». Рюкзак	22
Билет 3. Метод ветвей и границ. Дерево решений, границы, управляемый перебор	24
Билет 4. Приближённый алгоритм. Отклонение от оптимума. FPTAS для рюкзака	26
Билет 5. Стохастический алгоритм. Вероятностный анализ. Минимальный разрез (Каргер)	28
Билет 6. Сведение (reduction). Классы P и NP. NP-полнота и неразрешимость	30

Кратчайшие пути между всеми парами — Флойда–Уоршелла

ПОСТАНОВКА

Дан ориентированный взвешенный граф $G = (V, E)$, $|V| = n$, веса $w(u, v) \in \mathbb{R}$ (допускаются отрицательные рёбра, но без отрицательных циклов). Найти $d(i, j)$ — длину кратчайшего пути — для **всех** n^2 упорядоченных пар (APSP).

ИДЕЯ

ДП по множеству **разрешённых промежуточных вершин**. Пусть $d_{ij}^{(k)}$ — длина кратчайшего пути $i \rightarrow j$, у которого все **внутренние** вершины лежат в $\{1, \dots, k\}$. Постепенно «разрешаем» вершины $1, 2, \dots, n$; ответ — $d^{(n)}$.

АЛГОРИТМ

1. **Инициализация** $d^{(0)}$: $d_{ii} = 0$; $d_{ij} = w(i, j)$, если есть ребро $i \rightarrow j$; иначе $d_{ij} = +\infty$.
2. **Тройной цикл** (порядок важен — k снаружи):

$$\text{for } k = 1..n : \text{ for } i : \text{ for } j : d_{ij} \leftarrow \min(d_{ij}, d_{ik} + d_{kj})$$

3. (опц.) матрица предков pxt_{ij} для восстановления самого пути.

ТЕОРЕМА · КОРРЕКТНОСТЬ

При отсутствии отрицательных циклов после прохода $k = n$ имеем $d_{ij} = \delta(i, j)$ (истинное кратчайшее расстояние). Рекуррента:

$$d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}).$$

ИДЕЯ ДОКАЗАТЕЛЬСТВА

Кратчайший путь $i \rightarrow j$ с внутренними вершинами в $\{1..k\}$ — это один из двух случаев: (а) он **не проходит** через $k \Rightarrow$ его внутренние вершины $\subseteq \{1..k-1\}$, значит длина $= d_{ij}^{(k-1)}$; (б) он **проходит** через k — без отрицательных циклов ровно один раз — и распадается на $i \rightarrow k$ и $k \rightarrow j$, у каждой части внутренние вершины $\subseteq \{1..k-1\}$. Минимум двух случаев и даёт рекурренту.

СТРОГОЕ ДОКАЗАТЕЛЬСТВО

Индукция по k . База $k = 0$: пути без внутренних вершин — это рёбра, и $d^{(0)}$ заполнено весами. Шаг: пусть утверждение верно для $k-1$. Возьмём оптимальный путь $p : i \rightsquigarrow j$ с внутренними вершинами $\subseteq \{1..k\}$; можно считать p простым (нет отрицательных циклов $\Rightarrow$ удаление цикла не увеличивает длину).

- Если $k \notin p$: внутренние $\subseteq \{1..k-1\}$, по предположению длина $= d_{ij}^{(k-1)}$.
- Если $k \in p$: вершина k встречается ровно раз, $p = p_1(i \rightsquigarrow k) \cdot p_2(k \rightsquigarrow j)$; внутренние p_1, p_2 лежат в $\{1..k-1\}$ и сами оптимальны (иначе укоротили бы p), по предположению их длины $d_{ik}^{(k-1)}$ и $d_{kj}^{(k-1)}$.

Минимум по случаям совпадает с рекуррентой $\Rightarrow d_{ij}^{(k)}$ корректно. При $k = n$ разрешены все вершины $\Rightarrow d_{ij}^{(n)} = \delta(i, j)$. $\square$

In-place корректность. На итерации k значения d_{ik} и d_{kj} не меняются (их обновление использовало бы $d_{kk} = 0$), поэтому единственный слой $n \times n$ даёт тот же результат, что и последовательность $d^{(k)}$.

СЛОЖНОСТЬ

$T_{\text{wc}}(n) = \Theta(n^3)$ — три вложенных цикла по n , тело $O(1)$, итого n^3 операций (не зависит от плотности). Память $\Theta(n^2)$ (in-place, один слой; +, $\Theta(n^2)$ на матрицу предков). Восстановление одного пути — $O(n)$.

ОГРАНИЧЕНИЯ ПРИМЕНИМОСТИ

Отрицательные рёбра — допустимы. Отрицательный цикл делает задачу некорректной; он **детектируется** как $d_{ii} < 0$ после прогона. Метод выгоден на **плотных** графах ($E \approx n^2$) и когда нужны **все** пары; для разреженных дешевле $n \times$ Дейкстра ($O(n(E + n \log n))$) или алгоритм Джонсона.

ПРИМЕР

Эталонный оргграф (слева) и эволюция матрицы расстояний (справа). Напр. $d_{24}^{(0)} = +\infty$, а разрешив вершину 1: $d_{24}^{(1)} = \min(\infty, d_{21} + d_{14}) = \min(\infty, 8 + 7) = 15$.

Эталонный оргграф (Флойд-Уоршелл)

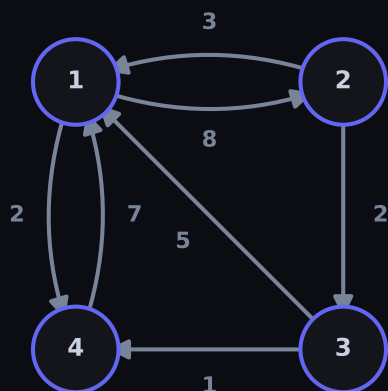


Рис. 1. Эталонный оргграф для прогона

Шаги Флойда-Уоршелла (жёлтым — обновлённые ячейки)

	1	$D^{(0)}$ — исходная			4		1	$D^{(1)}$ (через 1,2)			4		1	$D^{(4)}$ — итог			4
1	0	3	∞	7		1	0	3	5	7		1	0	3	5	6	
2	8	0	2	∞		2	8	0	2	15		2	5	0	2	3	
3	5	∞	0	1		3	5	8	0	1		3	3	6	0	1	
4	2	∞	∞	0		4	2	5	7	0		4	2	5	7	0	

Рис. 2. Слои $d^{(0)} \rightarrow d^{(1)} \rightarrow \dots$

НА ЭКЗАМЕНЕ

Цикл по k **обязан** быть внешним — перенос его внутрь ломает ДП. Аккуратно с $\infty + x$ (переполнение — используйте «большую» константу или проверку). Помните про детекцию отрицательного цикла ($d_{ii} < 0$) и восстановление пути через матрицу pxt.

ПОСТАНОВКА

Дана сеть $G = (V, E)$ с истоком s и стоком t и пропускными способностями $c(u, v) \geq 0$ (на отсутствующих рёбрах $c = 0$). Поток $f: E \rightarrow \mathbb{R}_{\geq 0}$ — функция, удовлетворяющая: ёмкости $0 \leq f(u, v) \leq c(u, v)$ и сохранению $\sum_v f(u, v) = \sum_v f(v, u)$ для всех $u \neq s, t$. Величина потока $|f| = \sum_v f(s, v) - \sum_v f(v, s)$. Найти поток максимальной величины.

ИДЕЯ

Остаточная сеть G_f : остаточная ёмкость $c_{f(u,v)} = c(u, v) - f(u, v)$ (плюс «обратные» рёбра $c_{f(v,u)} = f(u, v)$, позволяющие отменять поток). Увеличивающий путь — путь $s \rightsquigarrow t$ в G_f из рёбер с $c_f > 0$. Пока такой путь есть, проталкиваем по нему $\Delta = \min c_f$ и увеличиваем $|f|$ на Δ .

АЛГОРИТМ

1. Форд–Фалкерсон (обобщённый). Пока в G_f есть путь $s \rightsquigarrow t$: найти его (любым обходом), $\Delta \leftarrow \min_{e \in p} c_{f(e)}$, augment: $f(u, v) + = \Delta$, $f(v, u) - = \Delta$ вдоль пути.
2. Эдмондс–Карп. Тот же скелет, но путь — кратчайший по числу рёбер (BFS).
3. Диниц. Фазами: (1) BFS строит слоистую сеть L (уровни $\text{lev}(v) = \text{расстояние } s \rightarrow v \text{ в } G_f$, рёбра только $\text{lev}(u) + 1 = \text{lev}(v)$); (2) DFS находит блокирующий поток в L (с указателями «текущего ребра» и отсечением тупиков). Повторять, пока t достигим.

ТЕОРЕМА · КОРРЕКТНОСТЬ

(Max-Flow Min-Cut, Форд–Фалкерсон.) Для любого потока f эквивалентны: (1) f максимален; (2) в G_f нет увеличивающего пути; (3) $|f| = c(S, T)$ для некоторого s - t -разреза (S, T) . При этом $\max |f| = \min c(S, T)$. (Целочисленность.) Если все $c(u, v) \in \mathbb{Z}$, существует целочисленный максимальный поток (Ф.-Ф. строит именно его).

ИДЕЯ ДОКАЗАТЕЛЬСТВА

s - t -разрез (S, T) : $s \in S, t \in T$. Для любого потока $|f| = f(S, T) \leq c(S, T)$ (поток через разрез не больше его ёмкости) — это «слабая двойственность». Если увеличивающего пути нет, множество S вершин, достижимых из s в G_f , даёт разрез, на котором неравенство обращается в равенство $\implies$ обе величины оптимальны одновременно.

СТРОГОЕ ДОКАЗАТЕЛЬСТВО

Лемма (поток через разрез). Для любого разреза (S, T) справедливо $|f| = f(S, T) := \sum_{u \in S, v \in T} f(u, v) - \sum_{u \in S, v \in T} f(v, u)$. Доказательство: $|f| = \sum_v f(s, v) - \sum_v f(v, s)$; добавляя по сохранению нули $\sum_v f(u, v) - \sum_v f(v, u) = 0$ для всех $u \in S \setminus \{s\}$ и складывая, внутренние рёбра $S \rightarrow S$ сокращаются, остаётся сумма по рёбрам $S \rightarrow T$ и $T \rightarrow S$. Отсюда $|f| = f(S, T) \leq \sum_{u \in S, v \in T} c(u, v) = c(S, T)$ (поток неотрицателен, ёмкости не превышены). (*)

Эквивалентность (1) $\iff$ (2) $\iff$ (3).

- (1) $\implies$ (2): если есть путь, augment строго увеличивает $|f|$, противоречие.
- (2) $\implies$ (3): пусть $S = \{v : \exists \text{ путь } s \rightsquigarrow v \text{ в } G_f\}$, $T = V \setminus S$. Тогда $s \in S, t \in T$ (иначе путь — увеличивающий). Для рёбра $u \in S, v \in T$: $c_{f(u,v)} = 0 \implies f(u, v) = c(u, v)$; для $v \in T, u \in S$ обратного ребра в G_f нет $\implies f(v, u) = 0$. Подставляя в (*): $|f| = c(S, T)$.
- (3) $\implies$ (1): по (*) всякий поток $\leq c(S, T) = |f| \implies f$ максимален.

Целочисленность. Индукция по итерациям: $f \equiv 0$ цел; если f цел и c целы, то c_f целы $\implies \Delta$ цел $\implies$ новый f цел. Алгоритм завершается (величина растёт на ≥ 1 и ограничена), значит итоговый поток целочислен.

СЛОЖНОСТЬ

Форд–Фалкерсон (произвольный путь): $O(E \cdot f^*)$, где f^* — величина макс. потока; на иррациональных ёмкостях может вовсе не сходиться (контрпример Форда–Фалкерсона). Эдмондс–Карп (BFS): $O(V E^2)$ — каждое ребро становится «критическим» $O(V)$ раз, т.к. расстояние $\text{dist}(s, v)$ монотонно не убывает. Диниц: $O(V^2 E)$ — число фаз $O(V)$ (уровень t строго растёт), каждая фаза (блокирующий поток) $O(V E)$. На единичных ёмкостях Диниц даёт $O(E \sqrt{V})$.

ОГРАНИЧЕНИЯ ПРИМЕНИМОСТИ

Корректность и завершимость гарантированы для целых/рациональных ёмкостей (рациональные масштабируются до целых). Наивный Ф.-Ф. немонотонен по выбору пути: плохой выбор даёт $O(f^*)$ итераций (экспонента от размера входа) — поэтому на практике берут ЕК/Диниц. Антипараллельные рёбра требуют расщепления вершины или хранения остаточной ёмкости отдельно.

ПРИМЕР

Сеть с истоком s и стоком t . Минимальный разрез выделен: $c(S, T)$ = сумма ёмкостей рёбер $S \rightarrow T$ совпадает с найденной величиной потока $|f|$, что и иллюстрирует теорему о max-flow/min-cut. Обратные рёбра остаточной сети позволяют «перенаправить» уже посланный поток.

Сеть потока: $|f|_{\max} = 5$, мин. разрез $\{s\}|\{a,b,t\} = 5$

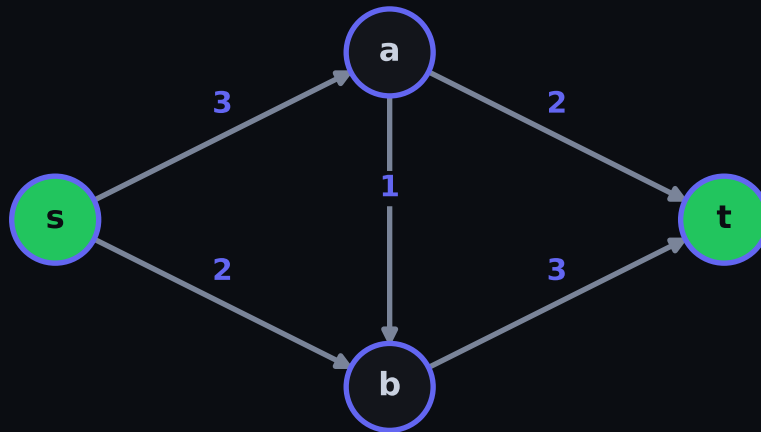


Рис. 3. Транспортная сеть: пропускные способности и поток

НА ЭКЗАМЕНЕ

Не путать **поток** (на рёбрах, с сохранением) и **разрез** (разбиение вершин). Величина потока считается **по любому** разрезу — удобно для проверки. На вопрос «почему BFS?» — отвечайте про монотонность dist и $O(VE^2)$. Помните: обратные рёбра нужны именно для **отмены** ошибочно посланного потока; без них жадность не находит оптимум.

Паросочетания в графе. Базовый алгоритм / Хопкрофта–Карпа

ПОСТАНОВКА

Паросочетание $M \subseteq E$ — множество попарно несмежных рёбер (никакие два не делят вершину). Вершина **насыщена**, если покрыта ребром из M . Различают: **максимальное по включению** (нельзя добавить ребро), **наибольшее по мощности** ($\max |M|$), **совершенное** (насыщены все вершины). Задача: найти наибольшее M (часто в **двудольном** графе $G = (L \cup R, E)$).

ИДЕЯ

Чередующийся путь относительно M — путь, рёбра которого попеременно $\notin M$ и $\in M$. **Увеличивающий путь** — чередующийся путь, оба конца которого **не насыщены**. Тогда он содержит на одно ребро вне M больше; **инвертировав** его ($M \leftarrow M \triangle p$), получаем паросочетание на единицу мощнее.

АЛГОРИТМ

1. **Базовый (алгоритм Куна, двудольный)**. Для каждой вершины $u \in L$ запускаем DFS/BFS поиск увеличивающего пути из u в текущей структуре; найдя — инвертируем его (мощность $+1$). Перебрав все u , получаем наибольшее M .
2. **Хопкрофт–Карп**. Работаем **фазами**:
 1. BFS от всех ненасыщенных $u \in L$ строит **слои** и кратчайшую длину ℓ увеличивающего пути;
 2. DFS набирает **максимальное по включению множество** вершинно-непересекающихся кратчайших ($= \ell$) увеличивающих путей и инвертирует их все разом.

Фазы повторяются, пока увеличивающие пути существуют.

ТЕОРЕМА · КОРРЕКТНОСТЬ

(Теорема Бержа.) Паросочетание M является наибольшим по мощности тогда и только тогда, когда относительно M не существует увеличивающего пути.

ИДЕЯ ДОКАЗАТЕЛЬСТВА

Если увеличивающий путь есть — M не максимально (инвертируем, $+1$). Обратно, если M не максимально, возьмём наибольшее M' , $|M'| > |M|$; в симметрической разности $M \triangle M'$ каждая вершина имеет степень ≤ 2 , поэтому компоненты — простые пути и чётные циклы. Подсчёт рёбер даёт путь с перевесом рёбер M' — это и есть увеличивающий путь относительно M .

СТРОГОЕ ДОКАЗАТЕЛЬСТВО

($\Leftarrow$) Контрапозиция: пусть есть увеличивающий путь p . Тогда $M' = M \triangle p$ — паросочетание (концы p не насыщены, внутренние вершины меняют партнёра) и $|M'| = |M| + 1$, значит M не наибольшее.

($\Rightarrow$) Пусть M не наибольшее, M' — наибольшее, $|M'| > |M|$. Рассмотрим $H = M \triangle M'$. Каждая вершина инцидентна ≤ 1 ребру из M и ≤ 1 из M' , значит $\deg_H \leq 2 \Rightarrow$ компоненты H суть **простые пути и циклы**, причём рёбра вдоль каждой компоненты чередуются $\frac{M}{M'}$. В цикле длина чётна $\Rightarrow$ поровну рёбер M и M' . Так как $|M'| > |M|$, рёбер M' в H больше, значит существует компонента-путь с **перевесом** рёбер M' ; она начинается и кончается ребром M' , оба конца не насыщены $M \Rightarrow$ это увеличивающий путь относительно M . $\square$

СЛОЖНОСТЬ

Базовый (Кун): $O(V)$ поисков увеличивающего пути по $O(E)$ каждый $\Rightarrow O(VE)$. **Хопкрофт–Карп**: число фаз $O(\sqrt{V})$ (после $\sqrt{V}$ фаз длина кратчайшего увеличивающего пути $\geq \sqrt{V}$, а таких непересекающихся путей $\leq \sqrt{V}$), каждая фаза $O(E) \Rightarrow T_{\text{wc}} = O(E\sqrt{V})$. Память $O(V + E)$.

ОГРАНИЧЕНИЯ ПРИМЕНИМОСТИ

Алгоритмы Куна и Хопкрофта–Карпа в чистом виде работают для **двудольных** графов (раскраска BFS-обходом; если граф не двудольен — есть нечётный цикл). В **общем** графе наивный поиск увеличивающего пути ломается на **нечётных циклах** — нужен алгоритм **цветков (Blossom)** Эдмондса, стягивающий нечётные циклы; он даёт наибольшее паросочетание за $O(V^3)$ (или $O(E\sqrt{V})$ в версии Micali–Vazirani).

ПРИМЕР

Двудольный граф $L \cup R$; жирным выделено текущее M , пунктиром — увеличивающий путь от ненасыщенной вершины слева к ненасыщенной справа. Инвертирование пути увеличивает $|M|$ на единицу; отсутствие такого пути (по Бержу) означает, что M — наибольшее.

Совершенное паросочетание (зелёным), $|M| = 3$

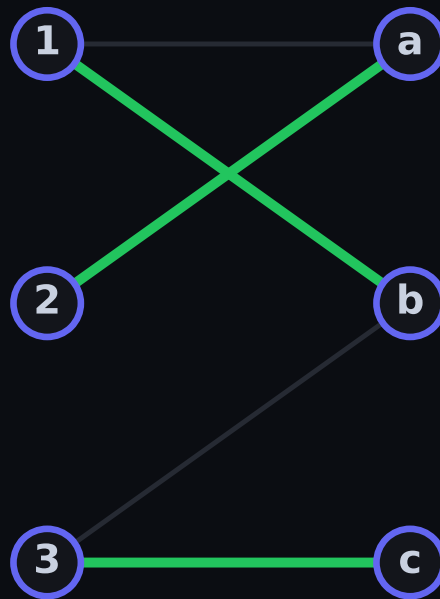


Рис. 4. Двудольное паросочетание и увеличивающий путь

НА ЭКЗАМЕНЕ

Чётко различайте **максимальное по включению** (локально, жадно) и **наибольшее по мощности** (глобально). Критерий Бержа — про **увеличивающий** (не просто чередующийся) путь: оба конца ненасыщены. Для двудольного графа задача сводится к **максимальному потоку** (единичные ёмкости): тогда НК = Диниц, отсюда $O(E\sqrt{V})$. На общий граф — только Blossom.

Правильная раскраска графа — жадный алгоритм

ПОСТАНОВКА

Правильная раскраска графа $G = (V, E)$ в k цветов — отображение $c : V \rightarrow \{1, \dots, k\}$, такое что $c(u) \neq c(v)$ для каждого ребра $\{u, v\} \in E$. **Хроматическое число** $\chi(G)$ — минимальное k , при котором правильная раскраска существует. Задача: построить правильную раскраску (по возможности малым числом цветов) и оценить $\chi(G)$.

ИДЕЯ

Жадный алгоритм: фиксируем порядок вершин $v_1, \dots, v_n$ и красим каждую в **наименьший** цвет, не встречающийся среди уже окрашенных соседей (минимальный свободный цвет, *tex*). Раскраска заведомо правильная; число использованных цветов зависит от порядка. **Уэлша–Пауэлла:** брать вершины в порядке **убывания степеней**.

АЛГОРИТМ

1. Упорядочить вершины $v_1, \dots, v_n$ (произвольно или по убыванию $\deg$).
2. Для $i = 1..n$: $c(v_i) \leftarrow$ наименьший цвет в $\{1, 2, \dots\}$, отсутствующий у уже окрашенных соседей v_i .
3. **Хроматический многочлен** $P(G, k)$ — число правильных раскрасок в k цветов; рекуррента **удаления–стягивания** по ребру e :

$$P(G, k) = P(G - e, k) - P(G/e, k),$$

где $G - e$ — удаление ребра, G/e — его стягивание. Тогда $\chi(G) = \min\{k : P(G, k) > 0\}$.

ТЕОРЕМА · КОРРЕКТНОСТЬ

Жадный алгоритм при **любом** порядке вершин использует не более $\Delta + 1$ цветов, где $\Delta = \max_v \deg(v)$ — максимальная степень; следовательно $\chi(G) \leq \Delta + 1$. При этом результат **зависит от порядка**: существует порядок, на котором жадный достигает $\chi(G)$, но плохой порядок может потребовать сильно больше цветов.

ИДЕЯ ДОКАЗАТЕЛЬСТВА

В момент окраски вершины v её уже окрашенные соседи занимают не более $\deg(v) \leq \Delta$ цветов, поэтому среди цветов $\{1, \dots, \Delta + 1\}$ хотя бы один свободен — жадный его и возьмёт. Значит ни одна вершина не получает цвет $> \Delta + 1$.

СТРОГОЕ ДОКАЗАТЕЛЬСТВО

Корректность раскраски. По построению $c(v)$ выбирается отличным от цветов всех **уже окрашенных** соседей; когда позднее красится сосед u , он, в свою очередь, избегает цвета $c(v)$. Значит для каждого ребра $\{u, v\}$ концы разноцветны. $\square$

Граница $\Delta + 1$ (индукция по номеру шага i). Утверждение: после шага i все цвета $c(v_1), \dots, c(v_i) \in \{1, \dots, \Delta + 1\}$. База $i = 0$ тривиальна. Шаг: при окраске v_i множество запрещённых цветов — это цвета её окрашенных соседей, их не более $\deg(v_i) \leq \Delta$ штук. Значит в наборе из $\Delta + 1$ цветов остаётся ≥ 1 свободный; жадный берёт наименьший свободный, т.е. цвет $\leq \Delta + 1$. По индукции граница держится для всех вершин $\Rightarrow \chi(G) \leq \Delta + 1$. $\square$

Зависимость от порядка. Для G существует **идеальный** порядок: раскрасим G оптимально в $\chi(G)$ цветов и упорядочим вершины по неубыванию их оптимального цвета — тогда жадный воспроизведёт ровно $\chi(G)$ цветов. Обратно, на короне («crown»/двудольном графе) подходящий «плохой» порядок заставляет жадный использовать $\approx \frac{n}{2}$ цветов вместо 2 — поэтому гарантии лишь $\Delta + 1$.

СЛОЖНОСТЬ

Жадный: $O(V + E)$ при разумной реализации (для каждой вершины перебираем соседей, находя *tex* за $O(\deg(v) + 1)$ с битовой/счётной отметкой; сумма степеней $= 2E$). Память $O(V + \Delta)$. **Вычисление $\chi(G)$** — NP-трудная задача (уже проверка $\chi(G) \leq 3$ NP-полна). **Хроматический многочлен** $P(G, k)$ — #P-трудно вычислять; рекуррента удаления–стягивания даёт $O(2^E)$ в худшем случае.

ОГРАНИЧЕНИЯ ПРИМЕНИМОСТИ

Жадный алгоритм **не оптимален и чувствителен к порядку**: $\Delta + 1$ может быть сильно завышено (для двудольного графа $\chi = 2$, а Δ велик). Эвристика Уэлша–Пауэлла улучшает практику, но гарантии оптимума не даёт. Точное $\chi(G)$ и $P(G, k)$ вычислимы лишь экспоненциально (NP-/#P-трудность). Теорема Брукса уточняет: $\chi(G) \leq \Delta$ для связного G , кроме полного графа K_n и нечётного цикла.

ПРИМЕР

Граф, раскрашенный жадно в порядке убывания степеней (Уэлша–Пауэлла). Каждая вершина получает минимальный свободный цвет относительно соседей; число цветов $\leq \Delta + 1$. Для K_4 ($\Delta = 3$) требуется ровно $4 = \Delta + 1$ цвета — граница достижима.

Жадная раскраска: $\chi(G) = 3$

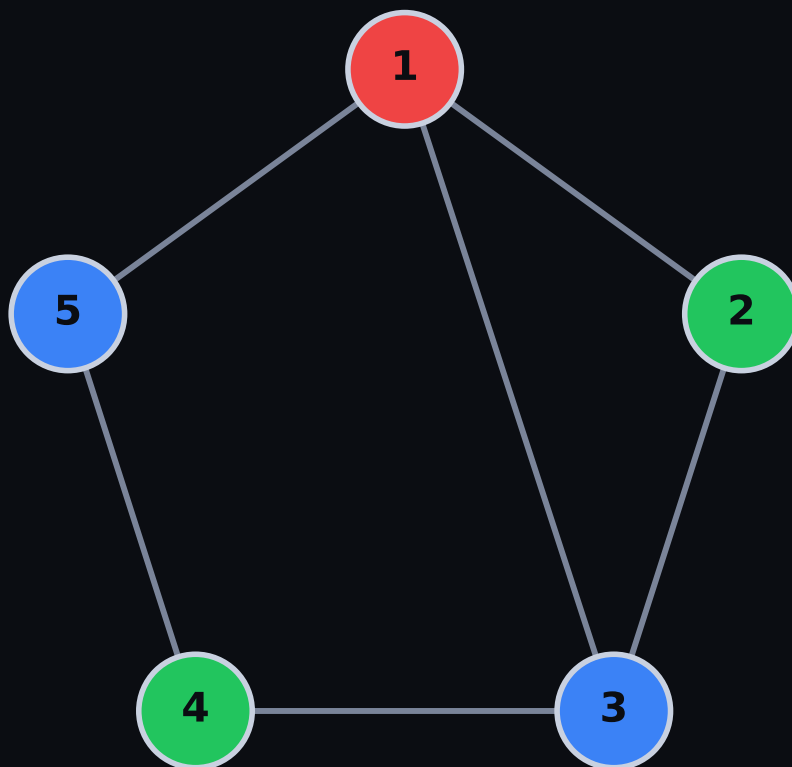


Рис. 5. Правильная раскраска: жадный обход по степеням

НА ЭКЗАМЕНЕ

Не путать **максимальную степень** Δ и **хроматическое число** χ : всегда $\chi \leq \Delta + 1$, но часто $\chi \ll \Delta$. Помните контрпример с «плохим порядком», где жадный далёк от оптимума. На вопрос «как точно?» — χ NP-трудно, $P(G, k)$ #P-трудно. Знак в рекурренте: $P(G, k) = P(G - e, k) - P(G/e, k)$ (**минус** стягивание), и упомяните теорему Брукса как усиление границы.

Точное вхождение строки в текст — Кнута–Морриса–Пратта / Бойера–Мура–Хорспула

ПОСТАНОВКА

Дан текст T длины n и образец P длины m над алфавитом Σ . Найти **все** позиции i , при которых P совпадает с подстрокой $T[i..i+m-1]$. Наивный перебор всех сдвигов — $O(nm)$; цель — убрать повторные сравнения.

ИДЕЯ

КМП. При несовпадении не откатывать указатель по тексту: уже прочитанный префикс образца «знает» свой длиннейший собственный бордер (одновременно префикс и суффикс), и сдвиг производится по **префикс-функции** π , а не на 1.

БМХ. Сравнить образец **справа налево** и при несовпадении прыгать по тексту далеко вперёд, используя символ текста, попавший на **последнюю** позицию окна (эвристика плохого символа). На больших алфавитах окно «перескакивает» сразу через много позиций.

АЛГОРИТМ

КМП. Префикс-функция $\pi[1..m]$: $\pi[q]$ — длина длиннейшего собственного бордера префикса $P[1..q]$.

1. Строим π за один проход (см. ниже).
2. Идём по T указателем i ; q — число уже совпавших символов образца. При несовпадении $T[i] \neq P[q+1]$ откатываем $q \leftarrow \pi[q]$ (повторяем, пока $q > 0$ и нет совпадения); при совпадении $q \leftarrow q+1$. Достигли $q = m$ — зафиксировали вхождение и сделали $q \leftarrow \pi[m]$.

БМХ (Хорспул). Таблица сдвига $\text{shift}[c] = m - 1 - \max\{j : P[j] = c, j < m - 1\}$, по умолчанию $\text{shift}[c] = m$ (символ не встречается левее последней позиции). Совмещаем окно, сравниваем справа налево; при несовпадении (или после совпадения) сдвигаем окно на $\text{shift}[c]$, где c — символ текста под **последней** клеткой окна.

ТЕОРЕМА · КОРРЕКТНОСТЬ

Оба алгоритма находят все вхождения. КМП работает за $O(n+m)$ в худшем случае; БМХ — за $O(nm)$ в худшем, но в среднем (большой случайный алфавит) делает $\approx O(\frac{n}{m})$ сравнений. Ключевая лемма КМП: $\pi[q]$ — длина **длиннейшего собственного бордера** $P[1..q]$, и итерация $q \leftarrow \pi[q]$ перебирает **все** бордеры по убыванию.

ИДЕЯ ДОКАЗАТЕЛЬСТВА

Корректность π . Все бордеры строки образуют цепочку, вложенную друг в друга: бордер бордера — снова бордер. Значит, отбросив длиннейший, следующий получаем как $\pi[\pi[q]]$ — поэтому при поиске можно безопасно «скользить» по failure-link, не пропуская ни одного возможного совмещения, на котором мог бы найтись образец.

СТРОГОЕ ДОКАЗАТЕЛЬСТВО

(1) **Бордеры вложены.** Пусть $b_1 > b_2$ — длины двух бордеров $P[1..q]$. Бордер длины b_1 означает $P[1..b_1] = P[q-b_1+1..q]$. Внутри этого равенства бордер длины $b_2 < b_1$ префикса $P[1..b_1]$ совпадает с суффиксом $P[1..q]$ той же длины, т.е. b_2 — тоже бордер $P[1..q]$. Следовательно множество бордеров — цепочка $q > \pi[q] > \pi[\pi[q]] > \dots > 0$, и итерация $q \leftarrow \pi[q]$ перечисляет их **все** по убыванию. Поэтому никакое более короткое совмещение, дающее совпадение, не пропускается $\Rightarrow$ найдены все вхождения.

(2) **Линейность КМП — амортизация потенциалом.** Возьмём потенциал $\Phi = q$ (число уже совпавших символов), $0 \leq q \leq m$. На каждом шаге внешнего цикла указатель текста i сдвигается ровно на 1 (n шагов всего). Каждое **успешное** сравнение увеличивает q на 1, т.е. $+1$ к Φ . Каждый откат $q \leftarrow \pi[q]$ строго **уменьшает** q (на ≥ 1), т.е. -1 или меньше к Φ . Поскольку Φ стартует с 0, не отрицателен и суммарно вырос не более чем на n (по числу успешных сравнений), суммарное число откатов также $\leq n$. Итого число операций $\leq 2n = O(n)$. Построение π — та же выкладка на P вместо T , $O(m)$. Итог $O(n+m)$. Указатель по тексту **никогда не откатывается назад**. $\square$

(3) **БМХ.** Сдвиг по плохому символу никогда не пропускает вхождение: символ c (символ под последней клеткой окна) должен где-то совпасть с образцом, и минимальный сдвиг, совмещающий ближайшее левое вхождение c в P с этой позицией, есть в точности $\text{shift}[c]$; меньший сдвиг заведомо оставит несовпадение в этой клетке.

СЛОЖНОСТЬ

КМП: $T = O(n+m)$, память $\Theta(m)$ (массив π). **БМХ:** препроцессинг $\Theta(m + |\Sigma|)$, поиск в худшем $O(nm)$ (напр. $T = a^n$, $P = a^m$ — каждое окно сравнивается целиком), в среднем при большом алфавите $\approx O(\frac{n}{m})$ (ожидаемый сдвиг порядка m). Память $\Theta(|\Sigma|)$ под таблицу сдвигов.

ОГРАНИЧЕНИЯ ПРИМЕНИМОСТИ

БМХ проседает на **малом алфавите** и **периодических** строках: сдвиги становятся короткими ($\rightarrow 1$), всплывает квадратичная оценка. **КМП** стабильно линеен **независимо** от алфавита и периодичности, но в среднем медленнее БМХ на «обычных» текстах с большим Σ . КМП к тому же даёт онлайн-обработку потока (один проход, без отката по тексту).

ПРИМЕР

Слева — построение π для образца **abacab** ($\pi = [0, 0, 1, 0, 1, 2]$); справа — один сдвиг Хорспула по последнему символу окна. Для **abacab**: при совпадении префикса **aba** и несовпадении на 4-й позиции откат $q \leftarrow \pi[3] = 1$, а не сброс в 0.

Префикс-функция КМР: образец «АВАВС»

i	1	2	3	4	5
P[i]	A	B	A	B	C
$\pi[i]$	0	0	1	2	0

Рис. 6. Префикс-функция π для **abacab**

Сдвиги Хорспула: образец «BARBER» (по P[0..4])

символ	B	A	R	E	проч.
сдвиг	2	4	3	1	6

Рис. 7. Сдвиг плохого символа БМХ

НА ЭКЗАМЕНЕ

Уметь за минуту посчитать π руками и объяснить, **почему указатель текста не откатывается** (потенциал $\Phi = q$).
Чёткое определение: $\pi[q]$ — длиннейший **собственный** бордер. Знать худший случай БМХ ($\frac{a^n}{a^m}$) и причину среднего $O(\frac{n}{m})$.

Приближённое вхождение — редакционное расстояние Левенштейна / Дамерау–Левенштейна

ПОСТАНОВКА

Даны строки A ($|A| = n$) и B ($|B| = m$). Найти **редакционное расстояние** — минимальное число элементарных операций, превращающих A в B . У Левенштейна это **вставка, удаление, замена** символа (цена 1); Дамерау добавляет **транспозицию** двух соседних символов.

ИДЕЯ

ДП по префиксам. $d[i, j]$ — расстояние между $A[1..i]$ и $B[1..j]$. Последняя операция оптимального выравнивания — одна из трёх (четырёх): её «снятие» сводит задачу к меньшим префиксам $\Rightarrow$ оптимальная подструктура.

АЛГОРИТМ

Левенштейн. База $d[i, 0] = i, d[0, j] = j$. Переход:

$$d[i, j] = \begin{cases} d[i-1, j-1] & \text{если } A_i = B_j \\ 1 + \min(d[i-1, j], d[i, j-1], d[i-1, j-1]) & \text{иначе} \end{cases}$$

где $d[i-1, j]$ — удаление A_i , $d[i, j-1]$ — вставка B_j , $d[i-1, j-1]$ — замена.

Дамерау (ограниченная). Добавить четвёртый кандидат при $A_i = B_{j-1} \wedge A_{i-1} = B_j$:

$$d[i, j] \leftarrow \min(d[i, j], d[i-2, j-2] + 1) \quad (\text{транспозиция}).$$

Восстановление выравнивания — обратный ход от $d[n, m]$ по выбранному кандидату.

ТЕОРЕМА · КОРРЕКТНОСТЬ

Рекуррента даёт точное редакционное расстояние: $d[n, m]$ — минимальная стоимость. Время $\Theta(nm)$, память $\Theta(\min(n, m))$ при хранении двух строк таблицы.

ИДЕЯ ДОКАЗАТЕЛЬСТВА

Любое выравнивание заканчивается **одной** операцией над последней позицией. Перебрав её тип и сняв, получаем оптимальное выравнивание префиксов меньшей длины; оптимум целого = (стоимость операции) + оптимум подзадачи. Минимум по типам и есть рекуррента.

СТРОГОЕ ДОКАЗАТЕЛЬСТВО

Оптимальная подструктура (Левенштейн). Рассмотрим оптимальную последовательность операций S , превращающую $A[1..i]$ в $B[1..j]$. Посмотрим на судьбу последнего столбца выравнивания:

- A_i удалён $\Rightarrow$ остаток превращает $A[1..i-1]$ в $B[1..j]$, и он оптимален (иначе заменили бы на лучший, уменьшив $|S|$) $\Rightarrow$ цена $= 1 + d[i-1, j]$.
- B_j вставлен $\Rightarrow$ аналогично $1 + d[i, j-1]$.
- A_i сопоставлен B_j : при $A_i = B_j$ цена 0 (совпадение) $+ d[i-1, j-1]$; иначе замена $1 + d[i-1, j-1]$.

Других вариантов для последнего столбца нет. Значит $d[i, j]$ равно минимуму перечисленного — это в точности рекуррента. **Индукция** по $i + j$ (база $i = 0$ или $j = 0$: нужно ровно i вставок / j удалений) даёт корректность всей таблицы. Для Дамерау добавляется случай «последние два символа переставлены» — $A_i = B_{j-1}, A_{i-1} = B_j$, снимаем транспозицию $\Rightarrow 1 + d[i-2, j-2]$. $\square$

СЛОЖНОСТЬ

$T = \Theta(nm)$ — таблица $n \times m$, обновление клетки $O(1)$. **Память.** Каждая строка зависит лишь от текущей и предыдущей $\Rightarrow$ хватает $\Theta(\min(n, m))$ (катим по короткой стороне). Для **ограниченной** Дамерау нужны **две** предыдущие строки. Восстановление выравнивания требует $\Theta(nm)$ памяти (полная таблица) либо приёма Хиршберга $\Theta(\min(n, m))$ памяти при $\Theta(nm)$ времени.

ОГРАНИЧЕНИЯ ПРИМЕНИМОСТИ

Restricted (ограниченная) Дамерау запрещает редактировать однажды транспонированную подстроку — это **не метрика** (нарушает неравенство треугольника); **полная** Дамерау–Левенштейна — корректная метрика, но дороже (учёт алфавита). Главное ограничение — **квадратичность**: для длинных строк применяют отсечение по полосе (band, при ограниченном $k = O(kn)$), битовые приёмы (Майерс, $O(n \frac{m}{w})$) или фильтрацию по q -граммам.

ПРИМЕР

Таблица ДП для $A = \text{кот}$, $B = \text{скат}$: путь правок $d = \text{кот} \rightarrow \text{скат}$ = вставка s + замена $o \rightarrow a = 2$; обратный ход по таблице восстанавливает само выравнивание (стрелки — выбранный кандидат минимума).

ДП Левенштейна: CAT → CART = 1 (вставка R)

	∅	C	A	R	T
∅	0	1	2	3	4
C	1	0	1	2	3
A	2	1	0	1	2
T	3	2	1	1	1

Рис. 8. Таблица $d[i, j]$ и обратный ход выравнивания

НА ЭКЗАМЕНЕ

Назвать смысл каждого из трёх соседей (↑ удаление, ← вставка, ↖ замена/совпадение). Помнить базу $d[i, 0] = i$. Отличать ограниченную Дамерау (не метрика) от полной. Оптимизация памяти до $\Theta(\min(n, m))$ – частый доп-вопрос.

Вхождение множества строк — автомат Ахо–Корасик

ПОСТАНОВКА

Дан набор образцов $P_1, \dots, P_r$ суммарной длины $m = \sum |P_i|$ и текст T длины n . Найти **все** вхождения **всех** образцов за один проход по тексту. Запуск КМП r раз дал бы $O(rn)$; цель — $O(n + m + z)$, где z — число найденных вхождений.

ИДЕЯ

Объединить образцы в **бор** (trie). При чтении текста двигаться по бору; на несовпадении — переходить по **суффиксной ссылке** (fail-ссылке), как failure-link в КМП, но для множества. **Выходные ссылки** перечисляют все образцы, заканчивающиеся в текущей вершине.

АЛГОРИТМ

- Бор.** Вставляем все P_i ; рёбра помечены символами, концевые вершины помнят свои образцы.
- Fail-ссылки (BFS по слоям).** Для корня и его детей fail = корень. Для вершины v с родителем u по символу c : идём по fail[u], fail[fail[u]], ... пока не найдём переход по c или не достигнем корня; туда и ведёт fail[v]. Корректно, т.к. родители обработаны **раньше** (меньшая глубина).
- Output-ссылки.** out[v] = v , если v концевая, иначе out[v] = out[fail[v]] — ближайший по fail-цепочке шаблон-суффикс.
- Поиск.** Идём по T ; при отсутствии перехода по c откатываемся по fail; в каждой вершине обходим output-цепочку и выдаём вхождения.

ТЕОРЕМА · КОРРЕКТНОСТЬ

fail[v] — вершина, соответствующая **длиннейшему собственному суффиксу** строки $s(v)$ (метки пути от корня до v), который является **префиксом** какого-то образца (т.е. присутствует в боре). Алгоритм находит все вхождения за $O(n + m + z)$.

ИДЕЯ ДОКАЗАТЕЛЬСТВА

Fail-ссылка — прямое обобщение префикс-функции: вместо «суффикс = префикс той же строки» здесь «суффикс = некоторый путь в боре». BFS гарантирует, что при вычислении fail[v] все более короткие суффиксы (на меньшей глубине) уже готовы, так что цепочку fail можно «доезжать» по уже построенным ссылкам.

СТРОГОЕ ДОКАЗАТЕЛЬСТВО

(1) Инвариант fail. Индукция по глубине v . База: глубина 1 — длиннейший собственный суффикс пуст, fail = корень. Шаг: пусть v — ребёнок u по символу c , $s(v) = s(u) \cdot c$. Любой непустой собственный суффикс $s(v)$ имеет вид $w \cdot c$, где w — суффикс $s(u)$. Длиннейший такой w , присутствующий в боре и имеющий c -переход, находится спуском по fail-цепочке u (по индукции она перечисляет суффиксы $s(u)$ по убыванию длины) до первой вершины с переходом по c . Эта вершина-цель и есть fail[v] — длиннейший суффикс $s(v)$, присутствующий в боре.

(2) BFS корректен. fail[v] ссылается только на вершины **меньшей** глубины (суффикс строго короче). BFS обрабатывает вершины по возрастанию глубины $\implies$ к моменту вычисления fail[v] все нужные fail-ссылки родительской цепочки уже определены.

(3) Полнота вывода. В позиции i автомат стоит в вершине v , где $s(v)$ — длиннейший суффикс $T[1..i]$, присутствующий в боре. Образец P заканчивается в $i \iff P$ — суффикс $s(v)$, лежащий в боре $\implies P$ достижим по output-цепочке из v . Значит обход output-ссылок выдаёт **ровно** все вхождения, оканчивающиеся в i . $\square$

СЛОЖНОСТЬ

Построение бора и fail/output — $O(m|\Sigma|)$ (или $O(m)$ на хеш-переходах). **Поиск:** указатель текста не откатывается (та же амортизация, что в КМП: потенциал вхождений по output-цепочкам. Итого $O(n + m + z)$. Память $\Theta(m|\Sigma|)$ для полного автомата (таблица переходов) или $\Theta(m)$ для разреженного бора.

ОГРАНИЧЕНИЯ ПРИМЕНИМОСТИ

Память — узкое место: **полный** автомат с предвычисленными переходами требует $O(m \cdot |\Sigma|)$ (для больших Σ , напр. Unicode, дорого; спасают хеш-таблицы / сжатые переходы). z может быть большим (много перекрывающихся образцов) — тогда сама **выдача** доминирует. Чувствителен к опечаткам: ищет только **точные** вхождения набора.

ПРИМЕР

Бор для набора {he, she, his, hers} с fail-ссылками (пунктир) и output-ссылками. На тексте ushers: дойдя до she, fail ведёт в he, а output-цепочка выдаёт сразу she и he.

Бор Ахо-Корасик {he, she, his, hers}
 зелёным пунктиром — fail-ссылки (she→he, hers→s)

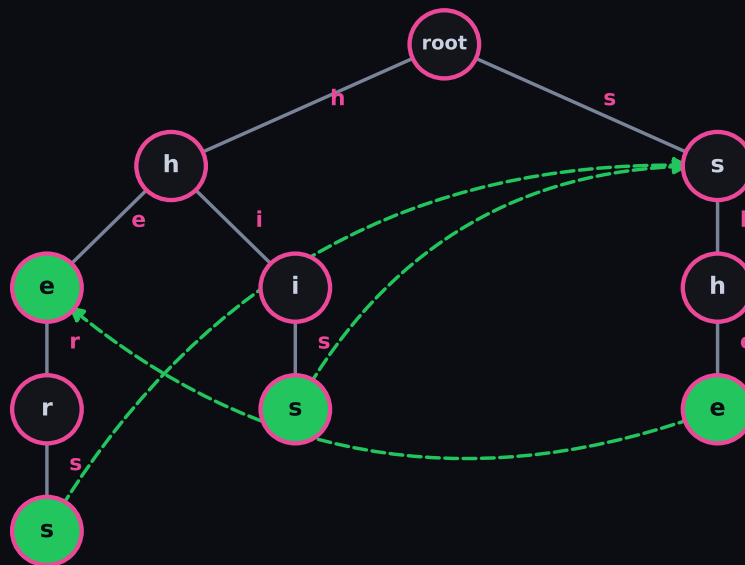


Рис. 9. Бор + fail-ссылки (пунктир) + output для {he,she,his,hers}

НА ЭКЗАМЕНЕ

Чётко: fail = длиннейший суффикс, **присутствующий в боре**; output = ближайший по fail концевой узел. Знать, **почему BFS** (fail указывает на меньшую глубину). Не забыть z в сложности — без выдачи поиск $O(n + m)$, с выдачей $+z$.

Специализированные сортировки строк — String Merge Sort / String Quick Sort / MSD Radix Sort

ПОСТАНОВКА

Отсортировать массив из n строк лексикографически. Обычная сортировка сравнениями тратит на каждое сравнение до $O(L)$ (L — длина строки) $\Rightarrow O(n \log n \cdot L)$. Цель — учитывать **посимвольную** структуру и общие префиксы, выразив стоимость через суммарную длину и длину **различающих префиксов** $D = \sum_i (\text{lcp-глубина } i)$.

ИДЕЯ

Сравнивать строки не целиком, а **по символам**, не перечитывая уже совпавшие префиксы. Три приёма: раскладка по символу позиции (MSD radix), тернарное разбиение по символу-пivоту (3-way string quicksort), слияние с переносом LCP (string mergesort).

АЛГОРИТМ

MSD radix sort (старший разряд первым).

1. На текущей позиции d разложить строки по d -му символу в $|\Sigma|$ корзин (строки, закончившиеся на позиции d , — в корзину «конец», она идёт первой).
2. Рекурсивно отсортировать **каждую** корзину по позиции $d + 1$.

3-way string quicksort. Выбрать пивот-символ $p = s[\text{mid}][d]$; разбить на три группы по d -му символу: $< p$, $= p$, $> p$. Рекурсия: $<$ и $>$ — по той же позиции d , $=$ — по позиции $d + 1$.

String mergesort. Обычное слияние, но храним LCP соседних строк; при слиянии сравнение начинаем с позиции $\min(\text{lcp})$, пропуская заведомо равные префиксы, и пересчитываем LCP для результата.

ТЕОРЕМА · КОРРЕКТНОСТЬ

Все три дают лексикографический порядок. Стоимость выражается через D — суммарную длину **различающих префиксов** (символов, реально просмотренных до различения): MSD и 3-way string quicksort работают за $O(D + n \log n)$ (quicksort — в среднем), что **не зависит** от хвостов строк за точкой различения.

ИДЕЯ ДОКАЗАТЕЛЬСТВА

Любой алгоритм сравнения обязан прочесть различающий префикс каждой строки хотя бы раз ($\Omega(D)$ — нижняя граница). Разрядные / тернарные методы **не перечитывают** совпавшие префиксы (рекурсия уходит вглубь по позиции), достигая $O(D)$ на работе с символами; $n \log n$ добавляет балансировка разбиений.

СТРОГОЕ ДОКАЗАТЕЛЬСТВО

Корректность MSD. Индукция по глубине d . После раскладки по d -му символу корзины упорядочены по этому символу; внутри корзины все строки имеют общий префикс длины d , и рекурсия по $d + 1$ упорядочивает их по остатку. Конкатенация корзин в порядке символов даёт лексикографический порядок (строки-«концы» короче и идут первыми — корректно, т.к. префикс лексикографически меньше своих продолжений).

Оценка работы. Каждый просмотренный символ строки s_i — это символ внутри её различающего префикса (как только префикс стал уникальным, строка попадает в одноэлементную корзину, и рекурсия по ней останавливается). Суммарно просмотров символов = $O(D)$. Накладные расходы на создание/обход корзин и (для quicksort) разбиения дают $O(n \log n)$ при сбалансированных разбиениях. Итого $O(D + n \log n)$. String mergesort за счёт LCP не сравнивает уже совпавшие префиксы повторно $\Rightarrow$ та же $O(D + n \log n)$, при этом **устойчив**. $\square$

СЛОЖНОСТЬ

MSD radix: $O(D + n \log n)$ среднее (или $O(D + n|\Sigma|)$ с распределением по $|\Sigma|$ корзинам на каждом уровне); память $\Theta(n + |\Sigma|)$. **3-way string quicksort:** $O(D + n \log n)$ в среднем, $O(D + n^2)$ худший (плохой пивот), на месте, память $O(\log n)$ на стек. **String mergesort:** $O(D + n \log n)$ гарантированно, устойчив, доп-память $\Theta(n)$. Сравните с $O(n \log n \cdot L)$ у наивной сортировки сравнениями.

ОГРАНИЧЕНИЯ ПРИМЕНИМОСТИ

MSD несёт **накладные расходы на корзины**: на малых подмассивах выгоднее переключаться на простую вставку/3-way quicksort (как в практических реализациях). **3-way quicksort неустойчив** и имеет квадратичный худший случай. Все методы выигрывают на **длинных общих префиксах** (малое $\frac{D}{\text{суммарная длина}}$) и **умеренном** $|\Sigma|$; при огромном алфавите массив корзин MSD раздут.

ПРИМЕР

Трасса MSD для $\{abc, abd, ab, bca\}$ по позиции $d = 1$:

- корзина a: $\{abc, abd, ab\}$; корзина b: $\{bca\}$.
- Рекурсия a по $d = 2 \rightarrow$ все имеют b; по $d = 3$: «конец» (ab) первым, затем c (abc), затем d (abd).

- Итог: $ab < abc < abd < bca$. Просмотрены только различающие префиксы ($ab/abc/abd$ различаются на 3-м символе) — хвостов нет, D мало.

НА ЭКЗАМЕНЕ

Уметь объяснить D (различающие префиксы) и нижнюю границу $\Omega(D)$. Назвать trade-off: MSD/3-way quicksort быстры, но quicksort неустойчив и квадратичен в худшем; mergesort устойчив и гарантирован, но $\Theta(n)$ памяти. Помнить про переключение на вставку для малых корзин.

ПОСТАНОВКА

Сжать строку S над алфавитом Σ без потерь: построить кодирование, из которого S восстанавливается точно, минимизируя длину. RLE кодирует серии; Хаффман — оптимальный **префиксный** посимвольный код; LZW — **словарный** код для повторов.

ИДЕЯ

RLE: серию из k одинаковых символов c заменить парой (k, c) . **Хаффман:** частым символам — короткие коды; жадно сливаем два **наименее частых** символа, строя бинарное дерево префиксного кода снизу вверх. **LZW:** поддерживать растущий словарь подстроки; на ходу выдавать код длиннейшей известной подстроки и добавлять её расширение.

АЛГОРИТМ**Хаффман.**

1. Поместить символы с частотами f_i в мин-кучу.
2. Пока > 1 узла: извлечь два минимальных x, y , создать узел с весом $f_x + f_y$ и детьми x, y , вернуть в кучу.
3. Коды — метки рёбер (0/1) на путях корень→лист.

LZW. Словарь = все одиночные символы. Читаем длиннейшую подстроку w , уже лежащую в словаре; выдаём её код; добавляем $w + c$ (следующий символ) как новую запись; продолжаем с c . Декодер восстанавливает словарь синхронно.

ТЕОРЕМА · КОРРЕКТНОСТЬ

(Оптимальность Хаффмана.) Среди всех **префиксных** кодов код Хаффмана минимизирует среднюю длину $L = \sum_i f_i \ell_i$. Любой префиксный код удовлетворяет **неравенству Крафта** $\sum_i 2^{-\ell_i} \leq 1$, и нижняя граница средней длины (на символ при f_i =вероятности p_i) — энтропия $H = -\sum_i p_i \log_2 p_i$.

ИДЕЯ ДОКАЗАТЕЛЬСТВА

Префиксный код $\Leftrightarrow$ листья бинарного дерева (неравенство Крафта — это условие существования такого дерева). В оптимальном дереве два **наименее частых** символа можно считать **братьями на максимальной глубине**; слив их, получаем меньшую задачу — это и делает жадность. Индукция по числу символов.

СТРОГОЕ ДОКАЗАТЕЛЬСТВО

Обменный аргумент. Пусть x, y — два символа с наименьшими частотами. Покажем: есть оптимальное префиксное дерево, где x, y — братья на максимальной глубине. Возьмём оптимальное дерево T^* и его двух листьев-братьев a, b на максимальной глубине (в полном бинарном дереве самый глубокий лист имеет брата). Так как x — минимальной частоты, $f_x \leq f_a$, и x не глубже a . Поменяем местами x и a : изменение средней длины

$$\Delta = (f_a - f_x)(\ell_x - \ell_a) \leq 0,$$

ведь $f_a \geq f_x$ и $\ell_a \geq \ell_x$ (a — на макс. глубине). Значит обмен **не ухудшает** код. Аналогично переносим y к b . Получили оптимальное дерево, где x, y — братья внизу.

Индукция. Слив x, y в символ z с $f_z = f_x + f_y$, получаем задачу на $|\Sigma| - 1$ символов. Для алфавита из 2 символов код Хаффмана (0/1) тривиально оптимален — база. По предположению Хаффман оптимален для z -задачи; средняя длина связана как $L(T) = L(T') + f_x + f_y$ (лист z глубины ℓ заменён двумя глубины $\ell + 1$, прирост = $f_x + f_y$ — **одинаков** для любого дерева, где x, y — братья). Минимизация $L(T') \Rightarrow$ минимизация $L(T) \Rightarrow$ Хаффман оптимален. $\square$

Неравенство Крафта. Для префиксного кода сопоставим коду c_i интервал длины $2^{-\ell_i}$ в $[0, 1)$; префиксность $\Rightarrow$ интервалы не пересекаются $\Rightarrow \sum 2^{-\ell_i} \leq 1$. Отсюда $L \geq H$ (Гиббс/Йенсен).

СЛОЖНОСТЬ

RLE: один проход $\Theta(n)$. **Хаффман:** подсчёт частот $\Theta(n)$ + построение дерева на куче $O(|\Sigma| \log |\Sigma|)$ (часто пишут $O(n \log n)$ для n символов алфавита); кодирование $\Theta(\text{длина выхода})$. **LZW:** $\Theta(n)$ при словаре на хеше/три (амортизированно $O(1)$ на символ). Память: Хаффман — дерево $\Theta(|\Sigma|)$; LZW — словарь до фикс. размера.

ОГРАНИЧЕНИЯ ПРИМЕНИМОСТИ

Хаффман требует **статистики** (частот) $\Rightarrow$ либо два прохода, либо передачу таблицы; на символ даёт целое число бит $\Rightarrow$ проигрывает арифметическому кодированию при сильно скошенных p_i . **RLE** бесполезен (даже раздувает) **без длинных серий**. **LZW** адаптивен (один проход, словарь не передаётся), но при разнородных данных словарь «не разгоняется»; исторически отягощён **патентом** (GIF) — ныне истёк.

ПРИМЕР

Слева — дерево Хаффмана для частот $\{a : 5, b : 2, c : 1, d : 1\}$: сливаем $c, d \rightarrow 2$, затем $c, d \rightarrow 4$, затем $c, d \rightarrow 9$; коды $a = 0$, $b = 10$, $c = 110$, $d = 111$. Справа — прогон LZW: на $ababab$ словарь пополняется $ab, ba, aba, \dots$ и повторы кодируются всё длиннее.

Дерево Хаффмана A:5 B:2 C:1 D:1 — итого 15 бит

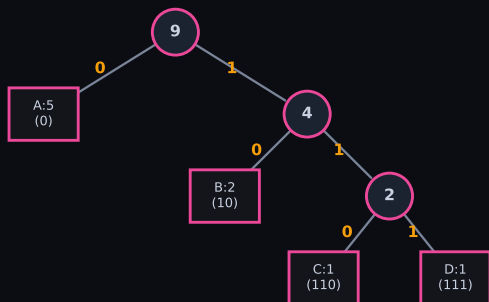


Рис. 10. Дерево Хаффмана (слияние двух минимальных)

LZW кодирование «АВАВАВА» → 1, 2, 3, 5

шаг	выдать	код	добавить
1	А	1	AB=3
2	В	2	BA=4
3	АВ	3	ABA=5
4	АВА	5	—

Рис. 11. Рост словаря LZW на ababab

НА ЭКЗАМЕНЕ

Обменный аргумент: два **наименее частых** — братья на **максимальной** глубине, далее индукция. Знать неравенство Крафта и границу $L \geq H$. Отличать статический Хаффман (нужна таблица) от адаптивного LZW (словарь синхронен у кодера/декодера).

Жадный алгоритм. Доказательство оптимальности**ПОСТАНОВКА**

Жадная стратегия: на каждом шаге делается **локально оптимальный** выбор — тот, что кажется наилучшим **сейчас**, без отката и пересмотра. Вопрос: когда последовательность таких выборов даёт **глобально оптимум**?

ИДЕЯ

Жадность корректна, если задача обладает двумя свойствами:

- **Свойство жадного выбора** (greedy-choice property): существует оптимальное решение, содержащее жадный выбор первого шага.
- **Оптимальная подструктура**: после фиксации жадного выбора остаётся подзадача того же типа, и её оптимум, дополненный жадным выбором, — оптимум исходной задачи.

Два инструмента доказательства: **обменный аргумент** (exchange argument) — превратить любой оптимум в жадный, не ухудшив; и **теория матроидов** — жадный алгоритм оптимален на $(E, \mathcal{I})$ **тогда и только тогда**, когда система независимых множеств $\mathcal{I}$ образует матроид.

АЛГОРИТМ

Общая схема (на примере **выбора заявок**, activity selection):

1. Отсортировать n объектов по «жадному критерию» (для заявок — по **времени окончания** f_i по возрастанию).
2. Идти по списку; брать очередной объект, если он **совместим** с уже выбранными (заявка $[s_i, f_i]$ не пересекается с последней взятой).
3. Вернуть набор.

Другие критерии: **дробный рюкзак** — по убыванию v_i/w_i ; **код Хаффмана** — сливать два **наименьших** по частоте узла; **MST** — Краскал (рёбра по возрастанию веса + DSU) либо Прим (расти дерево минимальным исходящим ребром).

ТЕОРЕМА · КОРРЕКТНОСТЬ

Жадный алгоритм выбора заявок (по раннему окончанию) возвращает множество **максимальной мощности** среди попарно совместимых заявок.

ИДЕЯ ДОКАЗАТЕЛЬСТВА

Принцип «жадный остаётся впереди» (staying ahead): после выбора k -й заявки её время окончания не позже, чем у k -й заявки **любого** допустимого решения. Раз жадный «не отстаёт» по времени, он успевает взять не меньше заявок.

СТРОГОЕ ДОКАЗАТЕЛЬСТВО

Обменный аргумент. Пусть жадное решение $G = (g_1, \dots, g_k)$ (в порядке окончания), а $O = (o_1, \dots, o_m)$ — произвольное оптимальное, тоже упорядоченное по f . Покажем индукцией, что $f(g_r) \leq f(o_r)$ для всех $r \leq k$.

- **База** $r = 1$: жадный берёт заявку с минимальным f из всех, значит $f(g_1) \leq f(o_1)$.
- **Шаг**: пусть $f(g_{r-1}) \leq f(o_{r-1})$. Заявка o_r совместима с o_{r-1} , т.е. $s(o_r) \geq f(o_{r-1}) \geq f(g_{r-1})$ — значит o_r совместима и с g_{r-1} , то есть была **кандидатом** для жадного на шаге r . Жадный выбрал g_r с минимальным f среди кандидатов $\implies f(g_r) \leq f(o_r)$.

Предположим $m > k$. Тогда в O есть заявка o_{k+1} с $s(o_{k+1}) \geq f(o_k) \geq f(g_k)$ — она совместима с g_k , поэтому жадный **не остановился бы** на шаге k , а взял g_{k+1} . Противоречие $\implies m = k$, жадный оптимален. $\square$

Матроидный взгляд. Если $\mathcal{I}$ — независимые множества матроида, а вес аддитивен, жадный по убыванию весов даёт максимально-весовую независимую базу (теорема Радо–Эдмондса). Для MST лес-матроид графа гарантирует оптимальность Краскала; для выбора заявок роль играет совместимость интервалов.

СЛОЖНОСТЬ

Выбор заявок: $O(n \log n)$ (сортировка) + $O(n)$ (проход) = $O(n \log n)$. Дробный рюкзак: $O(n \log n)$. Хаффман: $O(n \log n)$ (бинарная куча). Краскал: $O(E \log E)$; Прим с бинарной кучей: $O(E \log V)$.

ОГРАНИЧЕНИЯ ПРИМЕНИМОСТИ

Жадность **не** оптимальна там, где нет матроидной/обменной структуры. Классика — **0/1-рюкзак**: жадность по v_i/w_i ошибается. Контрпример: $W = 10$, предметы $(v, w) = (60, 6), (50, 5), (50, 5)$. Удельные ценности равны (10), жадность берёт предмет 1 (вес 6); остатка 4 не хватает ни на один из $(50, 5)$ — итог 60. Оптимум — два последних предмета: $50 + 50 = 100$ при весе ровно 10. Там, где жадность не работает, переходят к **ДП** или **В&Г**.

ПРИМЕР

Заявки (с,ф): $A(1, 4), B(3, 5), C(0, 6), D(5, 7), E(3, 9), F(5, 9), G(6, 10), H(8, 11)$. Сортировка по f : A, B, C, D, E, F, G, H . Жадно: берём $A(1, 4) \rightarrow$ следующая совместимая с $f = 4$ это $D(5, 7) \rightarrow$ затем $G(6, 10)$? нет ($6 < 7$), берём $H(8, 11)$. Ответ $\{A, D, H\}$ — три заявки, максимум.

НА ЭКЗАМЕНЕ

Всегда называйте **оба** свойства (жадный выбор + оптимальная подструктура) и **способ** доказательства. Обменный аргумент = «возьми чужой оптимум и превратите в жадный без потерь». Если просят «почему здесь жадность работает, а в 0/1-рюкзаке нет» — ключ в матроидной структуре.

Динамическое программирование: подходы. Связь с «разделяй-и-власт- вуй». Рюкзак

ПОСТАНОВКА

Динамическое программирование (ДП) решает задачу через **перекрывающиеся** подзадачи: значения подзадач вычисляются один раз и переиспользуются. Нужны два свойства: **оптимальная подструктура** и **перекрытие подзадач**.

ИДЕЯ

Два подхода к одной рекурренте:

- **Нисходящий** (top-down, мемоизация): обычная рекурсия + кэш; считаем только **достижимые** подзадачи, порядок задаёт стек вызовов.
- **Восходящий** (bottom-up, табличный): заполняем таблицу в порядке зависимостей (от базы к ответу); нет накладных расходов рекурсии, проще оценить память.

Отличие от «разделяй-и-властвуй»: там подзадачи **не пересекаются** (merge sort, бинарный поиск) — кэш бесполезен; в ДП подзадачи **общие**, и именно кэш убирает экспоненту. Naïve-рекурсия рюкзака/Фибоначчи без кэша экспоненциальна.

АЛГОРИТМ

0/1-рюкзак. Предметы $1..n$ с весами w_i , ценностями v_i , ёмкость W . $f(i, w)$ — максимальная ценность, набираемая из первых i предметов при лимите веса w . Рекуррента (берём i -й предмет или нет):

$$f(i, w) = \begin{cases} f(i-1, w) & \text{если } w_i > w \\ \max(f(i-1, w), f(i-1, w-w_i) + v_i) & \text{иначе} \end{cases}$$

База $f(0, w) = 0$. Ответ $f(n, W)$. Восходящее заполнение по $i = 1..n$, $w = 0..W$. **Восстановление набора:** идём от (n, W) назад; если $f(i, w) \neq f(i-1, w)$, предмет i взят, переходим в $(i-1, w-w_i)$, иначе в $(i-1, w)$.

ТЕОРЕМА · КОРРЕКТНОСТЬ

Восходящее заполнение даёт $f(n, W)$ = максимальной суммарной ценности допустимого (по весу $\leq W$) подмножества предметов.

ИДЕЯ ДОКАЗАТЕЛЬСТВА

Любое оптимальное подмножество либо **не содержит** предмет n (тогда это оптимум для $n-1$ предметов при том же W), либо **содержит** его (тогда остаток — оптимум для $n-1$ предметов при лимите $W-w_n$). Максимум двух вариантов.

СТРОГОЕ ДОКАЗАТЕЛЬСТВО

Индукция по i . База $i = 0$: предметов нет, $f(0, w) = 0$ — верно для всех w . Шаг: пусть $f(i-1, \cdot)$ корректно. Рассмотрим оптимальный набор $S \subseteq \{1, \dots, i\}$ для лимита w .

- Если $i \notin S$: $S \subseteq \{1, \dots, i-1\}$ допустим при лимите w и оптимален среди таких (иначе заменили бы), его ценность по предположению = $f(i-1, w)$.
- Если $i \in S$ (возможно лишь при $w_i \leq w$): $S \setminus \{i\}$ допустим при лимите $w-w_i$ для первых $i-1$ предметов и оптимален (иначе улучшили бы S), ценность = $f(i-1, w-w_i) + v_i$.

Оптимум — максимум достижимых случаев, что и есть рекуррента. Значит $f(i, w)$ корректно; при $i = n$, $w = W$ получаем ответ. $\square$

СЛОЖНОСТЬ

$T = \Theta(nW)$ — таблица $n \times W$, каждая ячейка за $O(1)$. Память $\Theta(nW)$, либо $\Theta(W)$ при прокатке одной строки (тогда w перебирать **по убыванию**, чтобы не использовать предмет дважды). Восстановление $O(n)$.

ОГРАНИЧЕНИЯ ПРИМЕНИМОСТИ

$\Theta(nW)$ **псевдополиномиально**: зависит от значения W , а не от размера входа ($\log W$ бит). При большом W (например $W = 2^{64}$) — практически неосуществимо, хотя n мало. 0/1-рюкзак NP-труден; полиномиального по длине входа ДП нет (если $P \neq NP$). Альтернатива при больших W — ДП по **ценности** $O(n^2 v_{\max})$ или приближение (см. билет 4).

ПРИМЕР

$W = 5$; предметы $(w, v) : (2, 3), (3, 4), (4, 5), (5, 6)$. Таблица $f(i, w)$ даёт $f(4, 5) = 7$ (взять предметы 1 и 2: вес $2 + 3 = 5$, ценность $3 + 4 = 7$), что лучше любого одиночного предмета.

ДП рюкзака 0/1, $W=7 \rightarrow$ оптимум 9 (предметы (3,4)+(4,5))

$i \setminus w$	0	1	2	3	4	5	6	7
$\emptyset$	0	0	0	0	0	0	0	0
(1,1)	0	1	1	1	1	1	1	1
(3,4)	0	1	1	4	5	5	5	5
(4,5)	0	1	1	4	5	6	6	9
(5,7)	0	1	1	4	5	7	8	9

Рис. 12. Таблица $f(i, w)$ и обратный проход для восстановления набора

НА ЭКЗАМЕНЕ

Чётко разведите **мемоизацию vs табличный** подход и **перекрытие подзадач** как границу с «разделяй-и-властвуй». На вопрос «полиномиален ли рюкзак» — твёрдо: **псевдополиномиален**, W входит линейно, но в битах это экспонента.

Метод ветвей и границ. Дерево решений, границы, управляемый перебор

ПОСТАНОВКА

Задача дискретной оптимизации (минимизация/максимизация на конечном, но экспоненциальном множестве). Полный перебор слишком дорог; нужен **направленный** перебор, отсекающий заведомо неперспективные варианты без потери оптимума.

ИДЕЯ

Перебор представляется **деревом решений**: узел — частичное решение, ветвление — фиксация очередной переменной (взять/не взять предмет, выбрать следующий город). Для каждого узла считается **граница** (bound) — **оптимистичная** оценка лучшего, что достижимо в его поддереве. Если граница узла **не лучше** текущего рекорда (incumbent), всё поддерево **отсекается** (pruning) — оно гарантированно не содержит улучшения. Порядок обхода (**управляемый перебор**: DFS / best-first по границе) влияет на скорость, но не на корректность.

АЛГОРИТМ

1. Инициализировать рекорд (incumbent) — любым допустимым решением или $-\infty$ (для максимума).
2. Поместить корень в очередь/стек. Пока не пусто:
 1. взять узел; если его **граница** $\leq$ рекорда (для максимума) — **отсечь**;
 2. если узел — лист (полное решение): обновить рекорд;
 3. иначе **ветвиться** — породить потомков, для каждого посчитать границу, добавить перспективных в очередь (best-first — сначала с лучшей границей).
3. Вернуть рекорд.

Границы на практике. Рюкзак: **дробная LP-релаксация** остатка (жадно по v/w , последний предмет дробно). Комми-вождёр: минимальное остовное дерево, сумма двух минимальных рёбер на вершину, 1-дерево. Задача о назначениях: **венгерский метод** (точное решение LP-релаксации) как нижняя оценка.

ТЕОРЕМА · КОРРЕКТНОСТЬ

Если функция границы **допустима** (для максимума: $\text{bound}(v) \geq$ значения любого листа в поддереве v), то ветви-границы возвращают **глобальный оптимум**: ни одно отсечение не теряет оптимальное решение.

ИДЕЯ ДОКАЗАТЕЛЬСТВА

Отсекается узел v лишь когда $\text{bound}(v) \leq$ рекорда. По допустимости все листья поддерева v не лучше $\text{bound}(v)$, значит не лучше рекорда — терять нечего. Оптимум выживает в неотсечённой ветви и будет найден.

СТРОГОЕ ДОКАЗАТЕЛЬСТВО

Пусть x^* — оптимум со значением ОПТ, и пусть рекорд в каждый момент $\leq$ ОПТ (инвариант: рекорд — значение реального допустимого решения). Рассмотрим путь от корня к листу x^* . Возьмём любой узел v на этом пути. Поскольку x^* лежит в поддереве v , по допустимости границы $\text{bound}(v) \geq \text{ОПТ} \geq$ рекорд. Условие отсечения — строгое $\text{bound}(v) <$ рекорд (или $\leq$ при поиске одного оптимума) — **не выполняется**, поэтому v **не отсекается**. Индукция по пути $\Rightarrow$ лист x^* достигается, рекорд обновляется до ОПТ. $\square$

СЛОЖНОСТЬ

В худшем случае дерево обходится целиком — $O(2^n)$ для рюкзака / $O(n!)$ для TSP-подобных задач: **гарантий полиномиальности нет**. На практике хорошие границы и удачный порядок отсекают подавляющую часть дерева. Стоимость узла = стоимость вычисления границы (для рюкзака — $O(n)$ на LP-релаксацию).

ОГРАНИЧЕНИЯ ПРИМЕНИМОСТИ

Эффективность **целиком** зависит от качества границы и порядка ветвления; плохая граница $\Rightarrow$ полный перебор. Память best-first — экспоненциальна (хранит фронт), DFS экономнее по памяти, но может «застыть». Метод **точный** (находит оптимум), в отличие от приближённых, но без верхней оценки времени.

ПРИМЕР

Рюкзак $W = 10$, предметы по убыванию v/w : $(v, w) : (40, 4), (42, 7), (25, 5)$. В корне дробная LP-граница: берём предмет 1 целиком (40, остаток 6), предмет 2 дробно ($6/7 \cdot 42 = 36$) $\Rightarrow$ граница 76. Ветвление «взять / не взять предмет 1»; узлы с границей $\leq$ текущего рекорда отсекаются.

Метод ветвей и границ: отсечение по границе

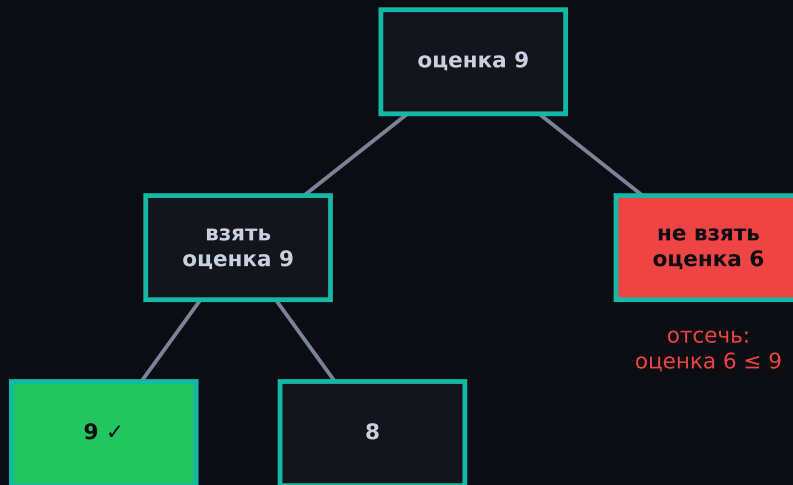


Рис. 13. Дерево решений рюкзака: ветвление, границы в узлах, отсечённые поддеревья

НА ЭКЗАМЕНЕ

Подчеркните: граница должна быть **оптимистичной** (верхняя для максимума), иначе теряется оптимум. «Управляемый перебор» = выбор порядка (best-first быстрее отсекает, DFS экономит память). Отличие от backtracking — наличие **числовой границы** для отсечения, а не только проверки допустимости.

Приближённый алгоритм. Отклонение от оптимума. FPTAS для рюкзака

ПОСТАНОВКА

Для NP-трудной задачи точное решение дорого. Приближённый алгоритм за полиномиальное время выдаёт решение с **гарантией** близости к оптимуму OPT.

ИДЕЯ

Коэффициент аппроксимации ρ : для максимизации $A(I) \geq \text{OPT}(I)/\rho$ (или $A \geq (1 - \varepsilon) \text{OPT}$); для минимизации $A \leq \rho \cdot \text{OPT}$. **Абсолютная** погрешность $|A - \text{OPT}|$ vs **относительная** $|A - \text{OPT}| / \text{OPT}$. Классы: **APX** — есть алгоритм с константным ρ ; **PTAS** (polynomial-time approximation scheme) — для любого $\varepsilon > 0$ даёт $(1 - \varepsilon)\text{OPT}$ за время, полиномиальное по n (но, возможно, экспоненциальное по $1/\varepsilon$); **FPTAS** — то же, но полиномиальное и по n , и по $1/\varepsilon$.

АЛГОРИТМ

FPTAS для 0/1-рюкзака (масштабирование ценностей). ДП по ценности работает за $O(n^2 v_{\max})$ — дорого при большом $v_{\max}$. Огрубим ценности:

1. $K = (\varepsilon \cdot v_{\max})/n$, где $v_{\max} = \max_i v_i$.
2. $v'_i = \lfloor v_i/K \rfloor$ — округлённые (меньшие) ценности.
3. Решить рюкзак **точно** ДП по ценности на v'_i ($O(n^2 v'_{\max})$).
4. Вернуть найденный **набор** предметов (с исходными ценностями v_i).

ТЕОРЕМА · КОРРЕКТНОСТЬ

Схема даёт допустимый набор S с $\sum_{i \in S} v_i \geq (1 - \varepsilon) \cdot \text{OPT}$ за время $O(n^3/\varepsilon)$ — это **FPTAS**.

ИДЕЯ ДОКАЗАТЕЛЬСТВА

Округление вниз теряет на каждом предмете $< K$ ценности, всего по набору $< nK = \varepsilon \cdot v_{\max} \leq \varepsilon \cdot \text{OPT}$. ДП на v' оптимально, поэтому оно не хуже округлённого истинного оптимума, а тот близок к OPT.

СТРОГОЕ ДОКАЗАТЕЛЬСТВО

Пусть S^* — оптимальный набор, $\text{OPT} = \sum_{i \in S^*} v_i$, и S — набор, выдаваемый ДП на округлённых v' . Из $v'_i = \lfloor v_i/K \rfloor$ следует $Kv'_i \leq v_i < Kv'_i + K$, то есть $v_i - K < Kv'_i \leq v_i$.

ДП оптимально по v' , значит $\sum_{i \in S} v'_i \geq \sum_{i \in S^*} v'_i$. Домножим на K и оценим:

$$\sum_{i \in S} v_i \geq K \sum_{i \in S} v'_i \geq K \sum_{i \in S^*} v'_i > \sum_{i \in S^*} (v_i - K) = \text{OPT} - |S^*| K.$$

Так как $|S^*| \leq n$ и $K = \varepsilon v_{\max}/n$, имеем $|S^*| K \leq nK = \varepsilon v_{\max}$. Наконец $v_{\max} \leq \text{OPT}$ (один самый ценный предмет помещается, иначе его исключаем). Значит $\sum_{i \in S} v_i > \text{OPT} - \varepsilon v_{\max} \geq (1 - \varepsilon)\text{OPT}$. $\square$

Время: $v'_{\max} = \lfloor v_{\max}/K \rfloor = \lfloor n/\varepsilon \rfloor$, ДП по ценности $O(n^2 v'_{\max}) = O(n^3/\varepsilon)$ — полином по n и $1/\varepsilon \implies \text{FPTAS}$.

СЛОЖНОСТЬ

$O(n^3/\varepsilon)$ времени, $O(n^2/\varepsilon)$ памяти (таблица ДП по ценности). Чем меньше ε , тем точнее и медленнее — управляемый компромисс **точность** $\leftrightarrow$ **время**.

ОГРАНИЧЕНИЯ ПРИМЕНИМОСТИ

Не у всех NP-трудных задач есть PTAS: для **APX-трудных** (напр. общий TSP, вершинное покрытие, max-3SAT) PTAS не существует, **если только** $P = NP$ (теорема PCP). Метрический TSP — 3/2-приближение (Кристофидес), но не PTAS. FPTAS — «лучший случай»; рюкзак относится к редким задачам, его имеющим.

ПРИМЕР

Пусть $v = (100, 60, 120)$, $n = 3$, $\varepsilon = 0.5$, $v_{\max} = 120$. Тогда $K = 0.5 \cdot 120/3 = 20$; $v' = (5, 3, 6)$. ДП на v' выбирает тот же выгодный набор, а гарантия $\sum v_i \geq 0.5 \cdot \text{OPT}$.



Рис. 14. Округление ценностей $v'_i = \lfloor v_i/K \rfloor$ и оценка потери $< \varepsilon \cdot \text{OPT}$

НА ЭКЗАМЕНЕ

Различайте **PTAS** (полином по n) и **FPTAS** (полином по n и $1/\varepsilon$). Ключ доказательства — суммарная потеря округления $< nK = \varepsilon v_{\max} \leq \varepsilon \text{OPT}$. Помните: $v_{\max} \leq \text{OPT}$ — без этой оценки гарантия не замкнётся.

Стохастический алгоритм. Вероятностный анализ. Минимальный разрез (Каргер)

ПОСТАНОВКА

Рандомизированный алгоритм использует случайные биты. Два класса: **Лас-Вегас** — всегда верный ответ, случайно лишь **время** (напр. randomized quicksort); **Монте-Карло** — фиксированное время, ответ верен с **вероятностью** (возможна ошибка). Алгоритм Каргера для **минимального разреза** — Монте-Карло.

ИДЕЯ

Глобальный минимальный разрез мультиграфа G — разбиение V на две непустые части с минимальным числом рёбер между ними. Идея Каргера: повторно **стягивать** (contract) случайное ребро (склеивая концы в одну вершину, оставляя кратные рёбра, удаляя петли), пока не останется 2 вершины — рёбра между ними образуют разрез-кандидат. Стянуть ребро фиксированного минимального разреза «невыгодно», поэтому он выживает с заметной вероятностью; многократный повтор делает вероятность ошибки сколь угодно малой.

АЛГОРИТМ

Contract(G):

- Пока $|V| > 2$: выбрать ребро e равновероятно среди всех рёбер; стянуть его (объединить концы, петли удалить, кратность сохранить).
- Вернуть число рёбер между двумя оставшимися супервершинами как кандидата.

Karger: запустить Contract $\Theta(n^2 \log n)$ раз, вернуть **минимум** кандидатов.

ТЕОРЕМА · КОРРЕКТНОСТЬ

Для фиксированного минимального разреза C (размера k) одна итерация Contract возвращает именно C с вероятностью

$$\Pr[C \text{ выжил}] \geq \frac{2}{n(n-1)} = \frac{1}{\binom{n}{2}}.$$

Поэтому $\Theta(n^2 \log n)$ независимых запусков находят минимальный разрез с вероятностью $1 - o(1)$.

ИДЕЯ ДОКАЗАТЕЛЬСТВА

C возвращается $\iff$ ни одно из его k рёбер ни разу не стянуто. На каждом шаге доля «плохих» рёбер мала (минимальность C ограничивает степени снизу), поэтому шаг безопасен с высокой вероятностью; перемножение телескопически даёт $1/\binom{n}{2}$.

СТРОГОЕ ДОКАЗАТЕЛЬСТВО

Пусть $|C| = k$. Если минимальный разрез равен k , то **степень** каждой вершины $\geq k$ (иначе разрез одной вершины был бы меньше), значит рёбер $|E| \geq nk/2$. На шаге i (когда осталось $n - i + 1$ вершин) вероятность стянуть ребро из C при условии, что оно ещё цело:

$$\Pr[\text{плохой шаг } i] = \frac{k}{|E_i|} \leq \frac{k}{(n-i+1)k/2} = \frac{2}{n-i+1}.$$

(на этом шаге у графа $n - i + 1$ вершин и минимальный разрез всё ещё $\geq k$). Вероятность **избежать** C на всех $n - 2$ шагах — произведение условных:

$$\Pr[C \text{ выжил}] \geq \prod_{i=1}^{n-2} \left(1 - \frac{2}{n-i+1}\right) = \prod_{j=3}^n \frac{j-2}{j}.$$

Телескоп: числитель $1 \cdot 2 \dots (n-2)$, знаменатель $3 \cdot 4 \dots n$, отношение

$$= \frac{1 \cdot 2}{(n-1) \cdot n} = \frac{2}{n(n-1)} = \frac{1}{\binom{n}{2}}.$$

Усиление. За $t = \binom{n}{2} \ln n$ независимых запусков вероятность ни разу не найти C :

$$\left(1 - \frac{1}{\binom{n}{2}}\right)^t \leq e^{-t/\binom{n}{2}} = e^{-\ln n} = \frac{1}{n} \rightarrow 0.$$

Значит $\Theta(n^2 \log n)$ повторов дают ошибку $\leq 1/n$. □

СЛОЖНОСТЬ

Одно стягивание — $O(n)$ (или $O(E)$); вся Contract — $O(n^2)$ (или $O(E\alpha)$ с DSU). Базовый Каргер: $O(n^2) \times \Theta(n^2 \log n) = O(n^4 \log n)$. **Каргер-Штайн** (рекурсия: стягивать до $n/\sqrt{2}$, ветвиться дважды) — $O(n^2 \log n)$ времени, вероятность успеха $\Omega(1/\log n)$ за запуск.

ОГРАНИЧЕНИЯ ПРИМЕНИМОСТИ

Монте-Карло — возможна ошибка (вернуть не-минимальный разрез): нужны повторы для надёжности; гарантии корректности лишь **вероятностные**. Детерминированные методы (Штор–Вагнер $O(n^3)$, или через max-flow) дают точный ответ без рандома. Анализ предполагает равновероятный выбор ребра среди **всех** (с учётом кратности) — реализация должна это соблюдать.

ПРИМЕР

Граф из двух «клик», соединённых 2 рёбрами (минимальный разрез $k = 2$). Одно стягивание может слить вершины **внутри** клики (безопасно) или **по мосту** (разрушит C). Базовая итерация сохранит C с вероятностью $\geq 1/\binom{n}{2}$, и повторы вытягивают её к 1.



Рис. 15. Стягивание рёбер до 2 супервершин; разрез-мост выживает с вероятностью $\geq 1/\binom{n}{2}$

НА ЭКЗАМЕНЕ

Разведите **Лас-Вегас** (точность гарантирована, время случайно) и **Монте-Карло** (наоборот). Сердце доказательства — оценка $|E| \geq nk/2$ через $\deg \geq k$, дающая $\Pr \leq 2/(n - i + 1)$ на шаг, и **телескопическое** перемножение до $1/\binom{n}{2}$. Назовите усиление: $\Theta(n^2 \log n)$ повторов $\implies$ ошибка $1/n$.

Сведение (reduction). Классы P и NP. NP-полнота и неразрешимость**ПОСТАНОВКА**

Формализовать «трудность» задач распознавания (decision problems, ответ да/нет $\equiv$ язык $L \subseteq \{0, 1\}^*$) через **вычислительные ресурсы** и через **сводимость** одной задачи к другой. Понять, какие задачи решаются эффективно, какие — лишь перебором, а какие **не решаются** в принципе.

ИДЕЯ

Сведение $A \xrightarrow{f} B$ — преобразование входа задачи A во вход задачи B так, что ответы совпадают. Тогда решатель B даёт решатель A : « B не легче A ». Полиномиальное сведение переносит **лёгкость вперёд** и **трудность назад** и задаёт иерархию сложности независимо от конкретного алгоритма.

АЛГОРИТМ

Ключевые определения.

- P** — языки, распознаваемые детерминированной машиной Тьюринга за время $O(n^k)$ при некотором k (эффективно решаемые).
- NP** — языки с **полиномиальным верификатором**: $x \in L \iff \exists y, |y| \leq \text{poly}(|x|)$, и $V(x, y) = 1$ за полином (y — сертификат). Эквивалентно: распознаются недетерминированной МТ за полином.
- Сведение по Карпу** $A \leq_p B$: функция f , вычисляемая за полином, с $x \in A \iff f(x) \in B$ (линейное — частный случай с f за $O(n)$).
- NP-трудная** задача H : $\forall L \in \text{NP} : L \leq_p H$. **NP-полная**: NP-трудная и $H \in \text{NP}$ — «самые трудные в NP».
- Неразрешимая** задача: не распознаётся **никаким** алгоритмом (при любом времени).

ТЕОРЕМА · КОРРЕКТНОСТЬ

- (Транзитивность.)** $A \leq_p B$ и $B \leq_p C \implies A \leq_p C$.
- (Кук–Левин.)** SAT (выполнимость булевой формулы) NP-полна.
- (Коллапс.)** Если хотя бы одна NP-полная задача лежит в P, то $P = \text{NP}$.
- (Неразрешимость.)** Проблема остановки HALT неразрешима.

ИДЕЯ ДОКАЗАТЕЛЬСТВА

Чтобы доказать NP-трудность **новой** задачи B , берут **известную** NP-полную A и строят сведение $A \leq_p B$ (направление: от трудной к новой!). Транзитивность тогда тащит на B всю NP. Коллапс $P = \text{NP}$ — это композиция двух полиномов: свести за полином и решить за полином.

СТРОГОЕ ДОКАЗАТЕЛЬСТВО

Транзитивность. Пусть f сводит A к B за время $p(n)$, g — B к C за $q(n)$. Композиция $g \circ f$ вычислима за $q(p(n)) + p(n)$ — полином (полином от полинома — полином); и $x \in A \iff f(x) \in B \iff g(f(x)) \in C$. $\square$

Коллапс. Пусть H NP-полна и $H \in P$ (решается за $q(n)$). Возьмём любой $L \in \text{NP}$. По NP-полноте $L \leq_p H$ некоторой f за $p(n)$. Тогда L решается так: по входу x вычислить $f(x)$ ($O(p(|x|))$), затем решить $H(f(x))$ ($O(q(p(|x|)))$) — суммарно полином. Значит $L \in P$, т.е. $\text{NP} \subseteq P$; включение $P \subseteq \text{NP}$ тривиально $\implies P = \text{NP}$. $\square$

Неразрешимость HALT (диагональ). Пусть алгоритм $H(M, x)$ решает, остановится ли M на x . Построим $D(M)$: «если $H(M, M) = \text{да}$ — заикнуться, иначе — стоп». Тогда $D(D)$ останавливается $\iff H(D, D) = \text{нет} \iff D(D)$ **не** останавливается — противоречие. Значит H не существует. $\square$

СЛОЖНОСТЬ

Известная иерархия: $P \subseteq \text{NP} \subseteq \text{PSPACE} \subseteq \text{EXP}$, причём $P \subsetneq \text{EXP}$ (теорема об иерархии по времени) — так что хотя бы одно включение строгое. Вопрос $P \stackrel{?}{=} \text{NP}$ **открыт**. Проверка сертификата в NP — полином; наивный перебор сертификатов — 2^{poly} .

ОГРАНИЧЕНИЯ ПРИМЕНИМОСТИ

NP-полнота — это **свидетельство практической трудности** в общем случае (если $P \neq \text{NP}$, полиномиального алгоритма нет). Реакция: приближённые алгоритмы и PTAS/FPTAS, эвристики и ветви-и-границы, параметризованная сложность, частные случаи. Внимание: **псевдополиномиальность** (рюкзак $\Theta(nW)$) не противоречит NP-полноте — это **слабая** NP-полнота. Неразрешимость $\neq$ NP-полнота: первое — про **вычислимость**, второе — про **эффективность**.

ПРИМЕР

Классические сведения (цепочки NP-полноты от Кука–Левина):

$$\text{SAT} \leq_p 3\text{-SAT} \leq_p \{\text{CLIQUE}, \text{VERTEX-COVER}, \text{IND-SET}\}, \quad 3\text{-SAT} \leq_p \text{HAM-CYCLE} \leq_p \text{TSP}, \quad \text{SUBSET-SUM} \leq_p \text{KNAPSACK}.$$

Карта классов при $P \neq \text{NP}$: P и NP-полные не пересекаются, между ними — NP-промежуточные (теорема Ладнера).

Классы сложности (в предположении $P \neq NP$)

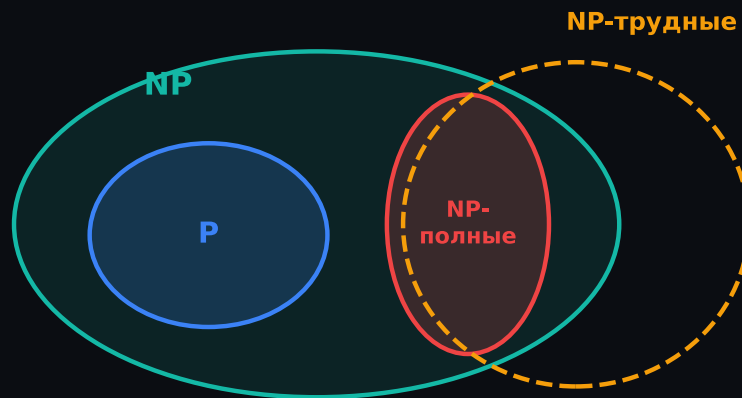


Рис. 16. Ландшафт сложности: $P \subseteq NP$, NP-полные задачи и предположение $P \neq NP$

НА ЭКЗАМЕНЕ

Не путать **NP-трудную** (всё из NP сводится к ней; может быть вне NP) и **NP-полную** (NP-трудная + сама в NP). Направление сведения для доказательства трудности — **от известной NP-полной к новой**: $A_{NP} \leq_p B$ (частая ошибка — наоборот). «NP» — это **не** «non-polynomial»; это полиномиальная **проверяемость**. $P \neq NP$ не доказано. Сертификат + верификатор — рабочее определение NP на экзамене.